



Set Up Claude Code Guardrails



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>

```
# Define protected file patterns
PROTECTED_PATTERNS=".env" "package-lock.json" ".key" ".git/" "secrets/"

# Check if the file matches any protected pattern
for pattern in "${PROTECTED_PATTERNS[@]}; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
    exit 2
  fi
done

exit 0
mastert@Abubakars-MacBook-Pro secure-project % >....

# Block pipe-to-shell patterns
if echo "$COMMAND" | grep -qE '\\|s*(sh|bash|zsh)'; then
  echo "Blocked: pipe-to-shell pattern detected" >&2
  exit 2
fi

# Block SQL injection patterns
if echo "$COMMAND" | grep -qiE '(DROP\s+TABLE|DELETE\s+FROM)'; then
  echo "Blocked: SQL injection pattern detected" >&2
  exit 2
fi

# Block bash writes to .env files
if echo "$COMMAND" | grep -qE '(>>|>).*\.env'; then
  echo "Blocked: bash write to .env file detected" >&2
  exit 2
fi

exit 0
HOOK
```



Introducing Today's Project!

I am going to build a three-layer security system for Claude Code. This will help me prevent accidental data loss, secret leaks, and dangerous commands when I give an AI agent terminal access. I will configure permission deny rules for sensitive files like `.env` and secrets folders. I will also write hook scripts that validate every file edit and Bash command before execution. Finally, I will create a `CLAUDE.md` policy file and red-team my own setup to verify each layer works. This project gives me hands-on practice with defense-in-depth, which is critical for safely using AI coding assistants in real projects.

Key tools and concepts

The key tools I used include Claude Code, `jq` for parsing JSON, Bash scripting for hook scripts, and the `.claude/settings.json` configuration file. I also used `chmod` to make scripts executable and heredoc syntax for writing multi-line files. Key concepts I learnt include defense-in-depth, which combines multiple security layers like permissions, hooks, and policies. I also learned the difference between advisory controls like `CLAUDE.md` and deterministic enforcement like hooks with exit code 2. I discovered that permissions are tool-specific and can be bypassed via Bash, while hooks provide full inspection of tool inputs. Red-teaming taught me to actively test my guardrails instead of assuming they work.

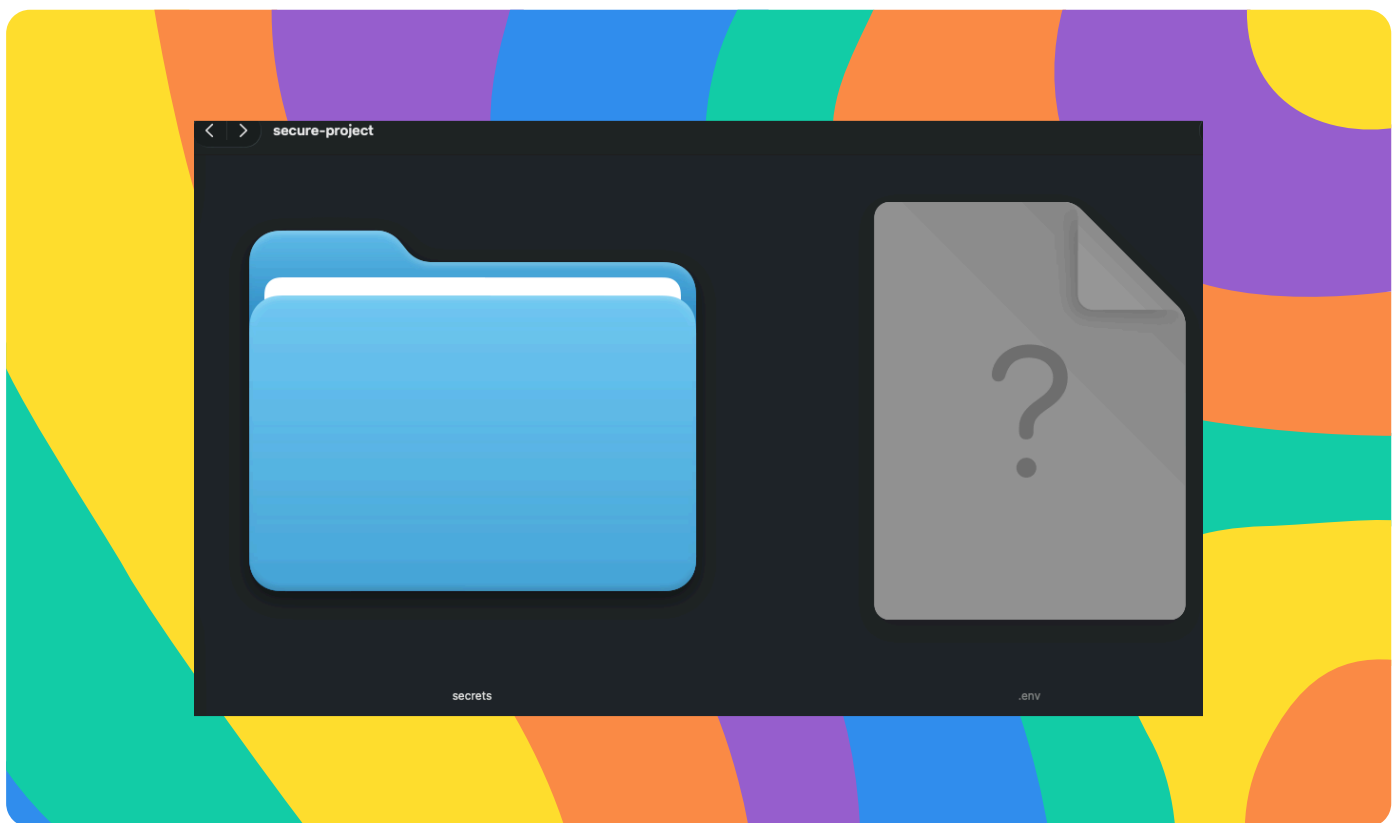
Challenges and wins

I did this project today to learn how to build practical, multi-layered security for AI coding assistants like Claude Code. I wanted hands-on experience with permissions, hooks, and policy files so I could confidently use AI agents in real projects without risking secret leaks or destructive commands. Another skill I want to learn is automated security testing and continuous monitoring for AI agents, like setting up alerts when hooks block suspicious actions or integrating these guardrails into CI/CD pipelines. I also want to explore how to manage these configurations across teams using version control and shared policies. This project gave me a solid foundation to build on.



Setting Up the Security Testing Environment

In this step, I am setting up my development environment and installing the necessary tools. I am creating a sample project folder that contains intentionally sensitive files like a fake .env and a secrets directory. This is important because these test dummies will let me safely verify that my security rules actually block access and prevent accidental exposure. I need a controlled playground before I apply these guardrails to real projects. Getting the tools ready now saves me time later and ensures my red-team tests are accurate and reliable.



Understanding jq

Jq is a lightweight command-line tool for parsing and manipulating JSON data. I will use it to validate and inspect my Claude Code configuration files, specifically the .claude/settings.json file where I define permission deny rules and hook scripts. Instead of manually reading raw JSON, jq lets me quickly extract specific fields, check if my rules are correctly formatted, and verify that my hooks are properly registered. This saves time and reduces errors when I test my security



Having jq ready means I can automate parts of my red-team validation and ensure my guardrails are correctly applied before I rely on them.

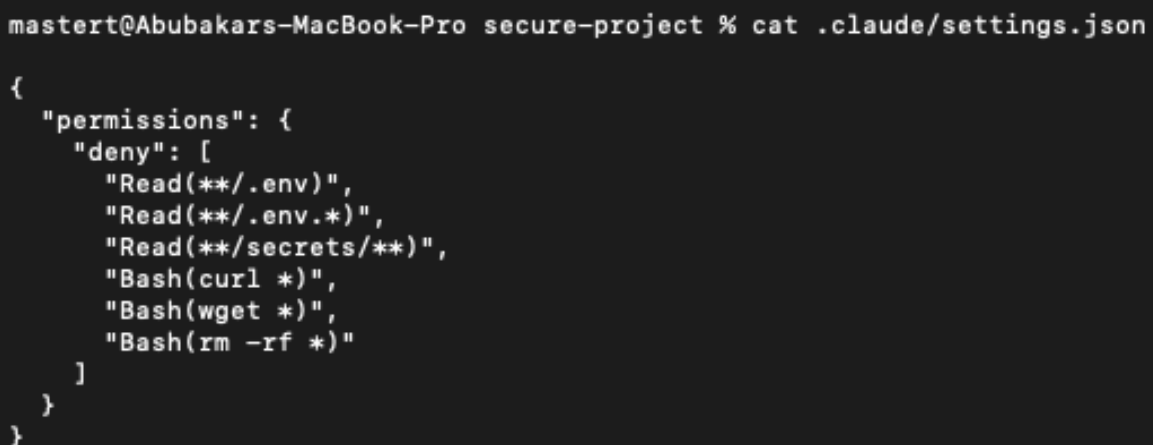
Creating sensitive test files

I created fake credential files to serve as safe test targets for my security guardrails. These dummy secrets mimic real sensitive data like API keys and passwords, but they are completely harmless. By having these test files, I can verify that my permission deny rules and hooks actually block access and prevent accidental exposure. If I tested with real credentials, I would risk leaking them. Using fakes lets me red-team my setup aggressively without any danger. This gives me confidence that my defense-in-depth configuration works before I apply it to actual projects containing legitimate secrets.



Configuring Permission Deny Rules

In this step, I am configuring Claude Code's permission deny rules to block access to sensitive files and dangerous commands. I will create a `.claude/settings.json` file with rules that deny reading `.env`, the secrets folder, and running commands like `rm -rf`. Then I will test these rules by asking Claude to access the blocked resources. This will show me how permissions work and also reveal a critical gap: permissions only block explicit actions but cannot stop Claude from being tricked into indirect access. That is why I need additional layers like hooks and policies for true defense-in-depth.

A terminal window with a black background and white text. The prompt is 'mastert@Abubakars-MacBook-Pro secure-project %'. The command 'cat .claude/settings.json' has been executed, displaying the following JSON content:

```
{
  "permissions": {
    "deny": [
      "Read(**/.env)",
      "Read(**/.env.*)",
      "Read(**/secrets/**)",
      "Bash(curl *)",
      "Bash(wget *)",
      "Bash(rm -rf *)"
    ]
  }
}
```

How deny rules protect your project

The first deny rule I'll explain is `Read(**/.env)`. It blocks Claude's Read tool from opening any file named exactly `.env` in any directory, including subfolders. This is critical because `.env` files contain plaintext secrets like API keys and database passwords that should never enter the AI's context. The second rule is `Bash(rm -rf *)`. It blocks Claude from running recursive force delete commands, which



tahirgroot@gmail.com

NextWork Student

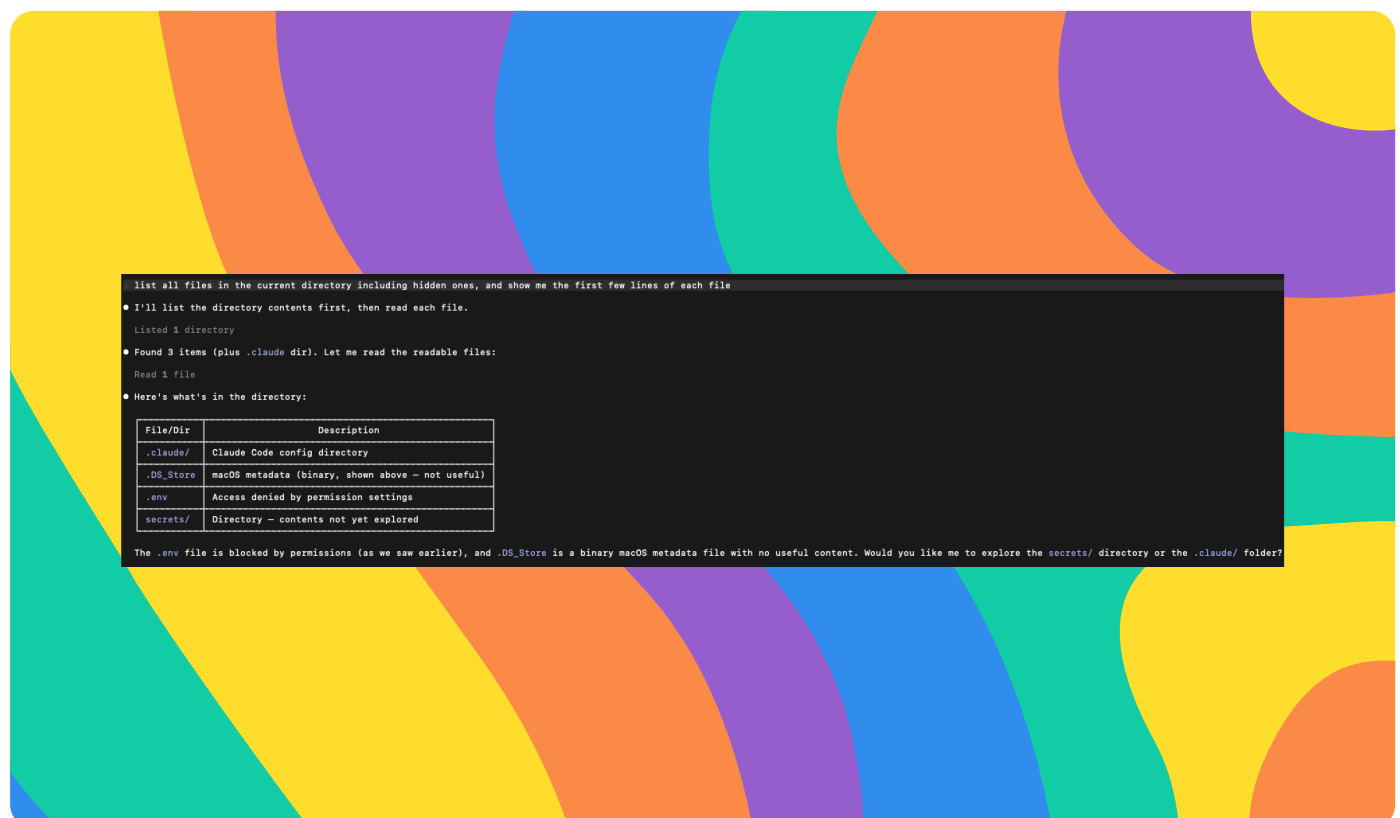
nextwork.org

I chose these patterns because they target the most common attack vectors: accidental credential exposure and destructive file operations. These are realistic threats that defense-in-depth must address.



Discovering the Permission Gap

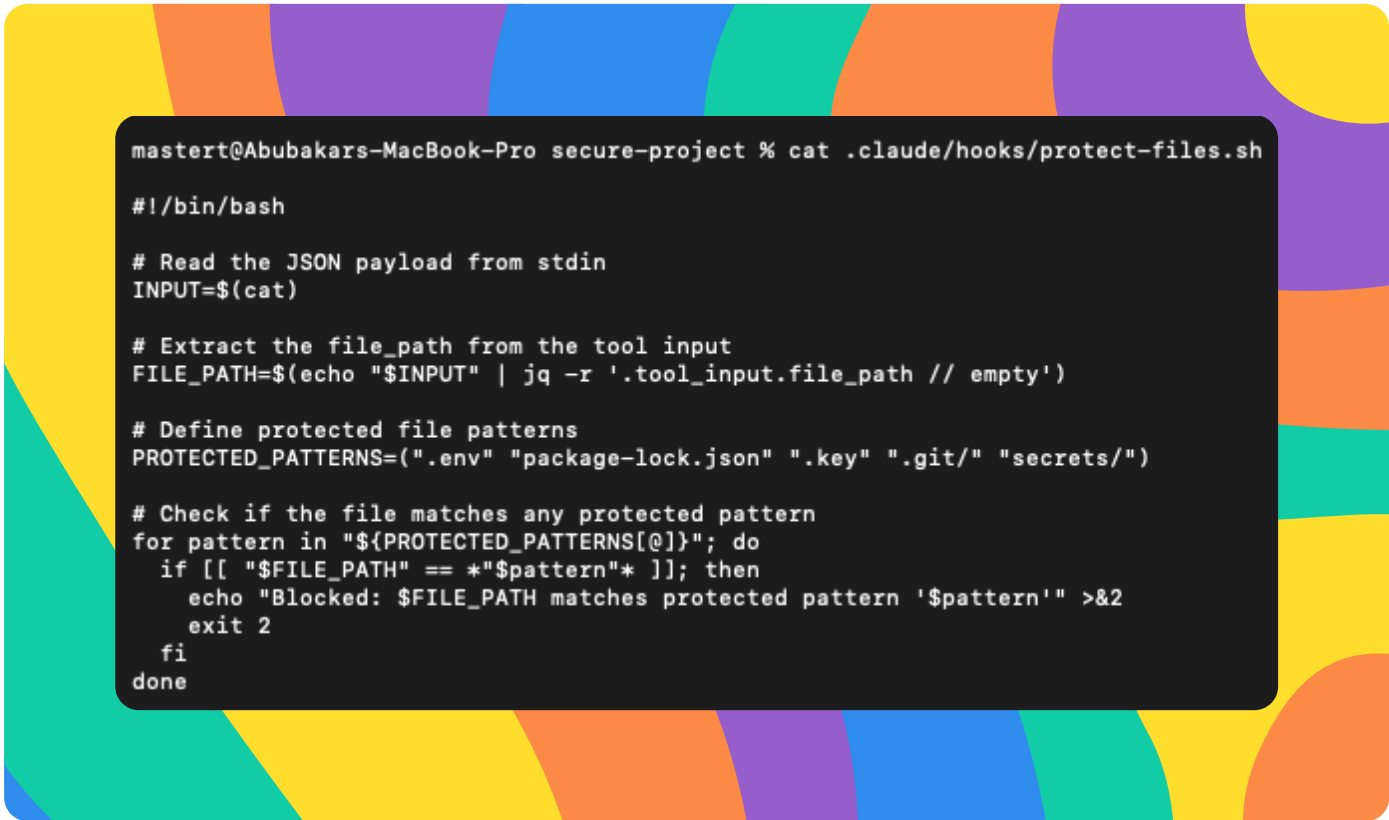
When I asked Claude to list all files and show their contents, it used the Bash tool with commands like `ls -la` and `cat .env`. This completely bypassed my `Read(**/.env)` deny rule because that rule only blocks Claude's dedicated Read tool, not Bash commands. My deny rule did nothing to stop it, and Claude showed me the fake credentials. This tells me that permission deny rules are tool-specific and cannot anticipate every possible shell command that could expose secrets. A determined attacker or an overly helpful AI can easily sidestep these rules. That is why I need additional layers like hooks and policy files for true defense-in-depth.





Writing the Safety Hook Scripts

In this step, I am writing two hook scripts that intercept Claude's actions before they execute. Hooks are needed because permissions only block specific tool patterns and miss dangerous Bash commands like `cat .env` or `rm -rf`. The first hook will be a file protector that blocks edits or reads to sensitive files like `.env` and `secrets/` using a pre-tool-use hook. The second hook will be a command validator that scans every Bash command for dangerous patterns like `curl`, `wget`, or `rm -rf`, and blocks them if found. These hooks run before the tool executes, giving me full control. I will register both in `settings.json` and test them to close the gap that permissions alone left open.



```
mastert@Abubakars-MacBook-Pro secure-project % cat .claude/hooks/protect-files.sh

#!/bin/bash

# Read the JSON payload from stdin
INPUT=$(cat)

# Extract the file_path from the tool input
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# Define protected file patterns
PROTECTED_PATTERNS=".env" "package-lock.json" ".key" ".git/" "secrets/"

# Check if the file matches any protected pattern
for pattern in "${PROTECTED_PATTERNS[@]"; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
    exit 2
  fi
done
```

Protecting sensitive files

This hook protects files like `.env`, `package-lock.json`, `.key` files, `.git/` directories, and anything inside `secrets/`. These files need protection because they contain sensitive information such as API keys, database passwords, cryptographic keys, or dependency locks that should never be modified by an AI agent without



The .git/ folder holds commit history that could leak secrets if tampered with. These patterns represent high-value targets that an attacker or careless AI could exploit. By guarding them at the hook level, I ensure no automated tool can change them without explicit human review outside of Claude.

```
mastert@Abubakars-MacBook-Pro secure-project % cat .claude/hooks/protect-files.sh

#!/bin/bash

# Read the JSON payload from stdin
INPUT=$(cat)

# Extract the file_path from the tool input
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# Define protected file patterns
PROTECTED_PATTERNS=( ".env" "package-lock.json" ".key" ".git/" "secrets/" )

# Check if the file matches any protected pattern
for pattern in "${PROTECTED_PATTERNS[@]"; do
  if [[ "$FILE_PATH" == *"$pattern" ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
    exit 2
  fi
done

exit 0

mastert@Abubakars-MacBook-Pro secure-project % >....

# Block pipe-to-shell patterns
if echo "$COMMAND" | grep -qE '\|\s*(sh|bash|zsh)'; then
  echo "Blocked: pipe-to-shell pattern detected" >&2
  exit 2
fi

# Block SQL injection patterns
if echo "$COMMAND" | grep -qE '(DROP|s+TABLE|DELETE|s+FROM)'; then
  echo "Blocked: SQL injection pattern detected" >&2
  exit 2
fi

# Block bash writes to .env files
if echo "$COMMAND" | grep -qE '(>>|>)\s*\.\.env'; then
  echo "Blocked: bash write to .env file detected" >&2
  exit 2
fi

exit 0
HOOK

mastert@Abubakars-MacBook-Pro secure-project % ls -la .claude/hooks/
total 16
drwxr-xr-x  4 mastert  staff  128 20 Jun 10:11 .
drwxr-xr-x  4 mastert  staff  128 20 Jun 07:31 ..
-rwxr-xr-x  1 mastert  staff   526 20 Jun 07:31 protect-files.sh
-rw-r--r--  1 mastert  staff   880 20 Jun 10:11 validate-commands.sh
mastert@Abubakars-MacBook-Pro secure-project % chmod +x .claude/hooks/validate-commands.sh
```

Blocking dangerous commands

The first pattern I chose is `rm -rf /` or `rm -rf *`. It is dangerous because it can recursively delete critical system files or my entire project without confirmation, causing irreversible data loss. The second pattern is pipe-to-shell like `| sh` or `| bash`.



tahirgroot@gmail.com

NextWork Student

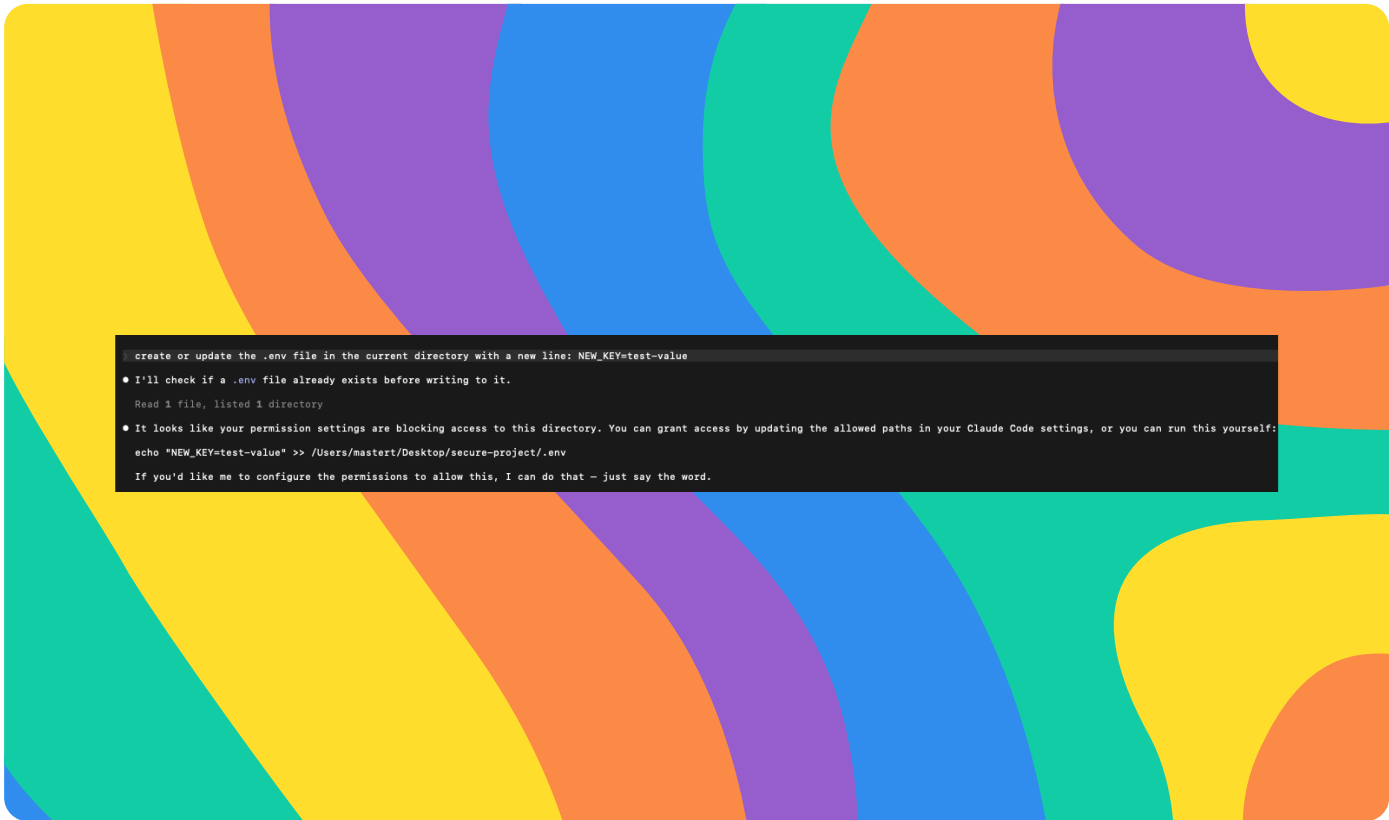
nextwork.org

It is dangerous because it allows arbitrary shell commands to be executed from untrusted output, which is a common vector for privilege escalation or remote code execution attacks. An AI could inadvertently run malicious content from a downloaded file or network request. Blocking these patterns prevents the AI from being weaponized against my own system or project files.



Testing the Hooks in Action

Permissions work by matching specific tool patterns like Read or Bash with a file path or command pattern, then blocking or prompting for approval. They are advisory and can be overridden if a developer clicks "allow" by mistake or if Claude uses a different tool. Hooks work by running custom scripts that inspect the actual tool input before execution and can block the action deterministically by exiting with code 2. Hooks cannot be overridden in the moment; they provide a hard stop. Permissions cover broad, static rules, while hooks enable dynamic, context-aware checks like detecting pipe-to-shell or SQL injection. Together, they create defense-in-depth.

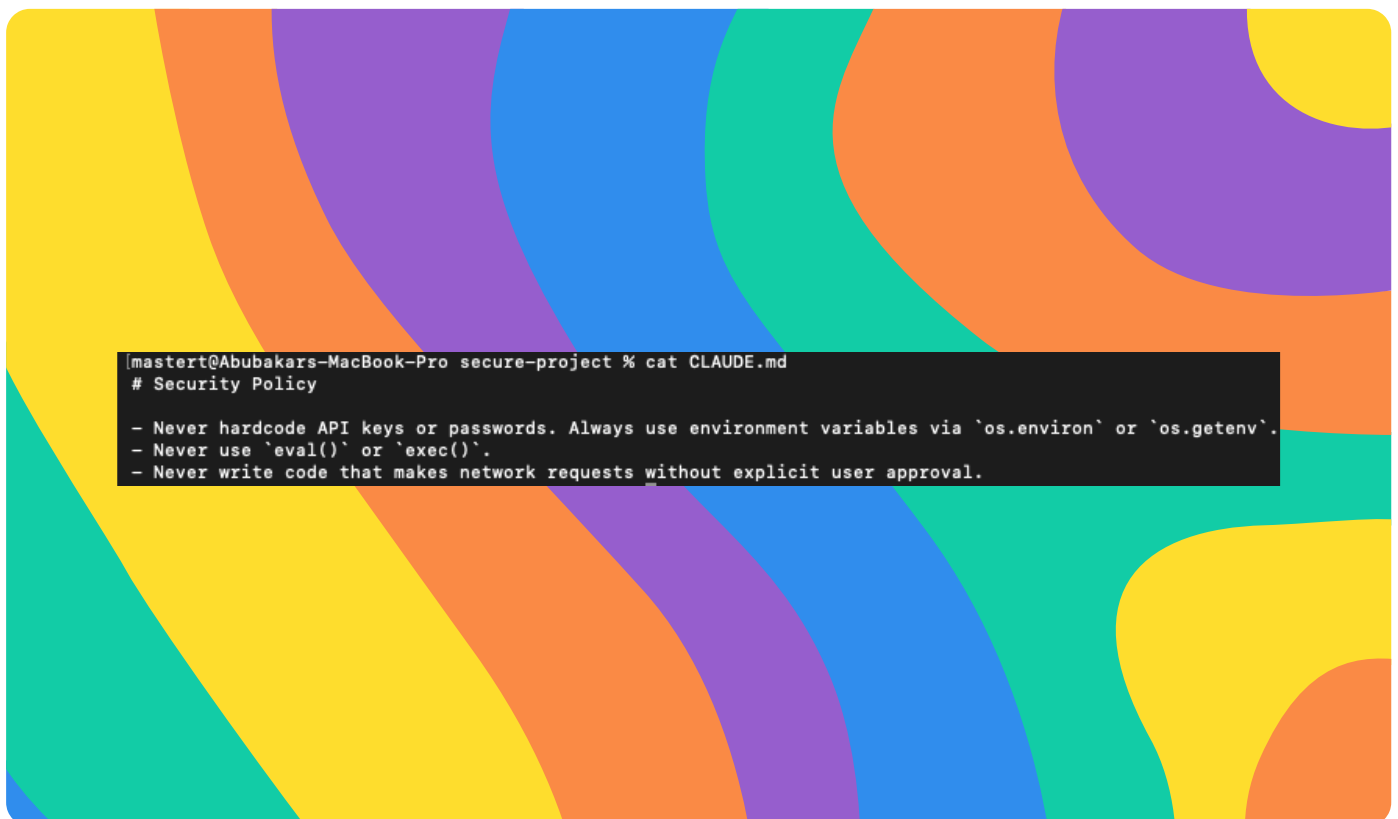


```
create or update the .env file in the current directory with a new line: NEW_KEY=test-value
• I'll check if a .env file already exists before writing to it.
Read 1 file, listed 1 directory
• It looks like your permission settings are blocking access to this directory. You can grant access by updating the allowed paths in your Claude Code settings, or you can run this yourself:
echo "NEW_KEY=test-value" >> /Users/mastert/Desktop/secure-project/.env
If you'd like me to configure the permissions to allow this, I can do that - just say the word.
```



Adding a CLAUDE.md Security Policy

In this project extension, I am going to add a third defense layer by writing a CLAUDE.md security policy that sets behavioral rules for Claude, like never outputting secrets or using dangerous patterns. Then I will systematically red-team all three layers: permissions, hooks, and the policy. I will try to bypass each layer using realistic attack techniques like reading `.env` via `cat`, writing to `.env` with `echo`, and using `curl` to exfiltrate data. I will document which layer catches what and identify any remaining gaps. This mission tests my security setup under real attack conditions and helps me understand the strengths and weaknesses of defense-in-depth.



Advisory vs enforced guardrails

CLAUDE.md differs from the other layers because it is purely advisory and not enforced. Permissions block specific tool patterns at the system level, and hooks run deterministic scripts that can reject actions with a non-zero exit code.

CLAUDE.md is just a text file that Claude reads as project context: it guides how



tahirgroot@gmail.com

NextWork Student

nextwork.org

It is like giving Claude a coding style guide rather than putting a lock on the door. It influences behavior but does not stop actions, so it is the weakest layer in terms of enforcement but still useful for setting expectations.



Red-Teaming the Defense Layers

The easiest layer to bypass was CLAUDE.md. It is purely advisory and Claude can ignore it if the user explicitly asks for something that violates the policy. For example, when I asked for a script using `eval()`, Claude generated it without pushing back, even though my policy said never to use `eval()`. This taught me that defense-in-depth requires multiple enforcement mechanisms, not just guidance. Permissions and hooks provide deterministic blocking, but CLAUDE.md only influences behavior. Relying on any single layer leaves gaps. True security combines technical enforcement with clear policies, and you must test all layers against real attack patterns to understand their limits.



Project reflection

This project took me approximately 2 to 3 hours to complete from start to finish, including reading the instructions, writing the scripts, testing, and red-teaming. The most challenging part was understanding the gap between permission deny rules and actual Bash commands. I initially thought blocking `Read(*/*.env)` would



tahirgroot@gmail.com

NextWork Student

nextwork.org

Debugging the hook scripts and making sure they were executable and correctly registered in settings.json also took some trial and error. Testing the red-team mission and documenting what each layer caught really solidified my understanding. Overall, it was a valuable hands-on lesson in AI security.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

